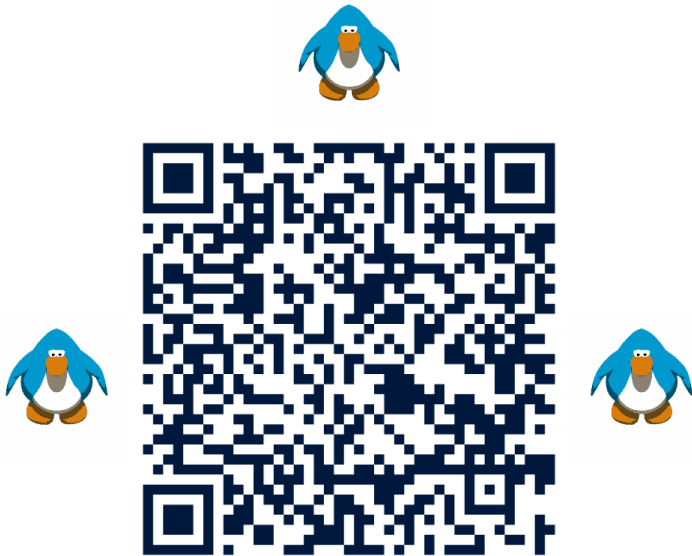


D2: Potential Method, Divide & Conquer

**Divide and conquer
algorithm when
multiply and
surrender walks in**



Sec 19: Th 4:30-5:30pm 2153 GGBL
IA: Anthony Chalhoub



Follow along with the
discussion worksheet!



Agenda

- ▶ Divide and Conquer

Divide and Conquer



Divide and Conquer: Key Ingredients

► Key Ingredients

- **Divide:** Divide a problem into **smaller versions** of the **same problem**
- **Conquer:** Combine the results from those subproblems
- **Base case:** When the input is small enough, the algorithm just directly output the answer

► Example: MERGESORT

```
MERGESORT ( $A[1, \dots, n]$ ):
```

```
  if  $n = 1$  then return  $A$ 
```

```
   $m \leftarrow \lfloor n/2 \rfloor$ 
```

```
   $L \leftarrow \text{MERGESORT}(A[1, \dots, m])$ 
```

```
   $R \leftarrow \text{MERGESORT}(A[m + 1, \dots, n])$ 
```

```
  return MERGE( $L, R$ ) //  $O(n)$ 
```

```
MERGE ( $L[1, \dots, \ell], R[1, \dots, r]$ ): // Recursive version
```

```
  if  $\ell = 0$  then return  $R$ 
```

```
  if  $r = 0$  then return  $L$ 
```

```
  if  $L[1] \leq R[1]$  then
```

```
    return  $L[1]: \text{MERGE}(L[2, \dots, \ell], R[1, \dots, r])$ 
```

```
  else
```

```
    return  $R[1]: \text{MERGE}(L[1, \dots, \ell], R[2, \dots, r])$ 
```

Divide and Conquer Correctness Proof

- Similar idea to prove by induction

Prove by Induction	Correctness Proof for D&C
Prove that $P(0)$ is true	Prove that the base case(s) is/are correct
Assuming $P(k)$ is true, prove $P(k + 1)$ is true	Assuming the outputs of the recursive calls are correct, prove that the final output (for the original input) is also correct Formally, this conclusion is reached by <i>strong induction</i> . See Handout 2 .

Divide and Conquer: General Form

- ▶ Consider an arbitrary divide-and-conquer algorithm that breaks a problem of size n into:
 - ▶ k smaller subproblems where $k \geq 1$
 - ▶ Each subproblem is of size n/b , where $b > 1$
 - ▶ The cost of splitting and combining results is $O(n^d)$ where $d \geq 0$
- ▶ This algorithm has the following recurrence
$$T(n) = kT\left(\frac{n}{b}\right) + O(n^d)$$
- ▶ We can apply the [Master Theorem](#) for runtime analysis

Master Theorem

- ▶ For the recurrence relation $T(n) = kT\left(\frac{n}{b}\right) + O(n^d)$, $k \geq 1$, $b > 1$, $d \geq 0$

$$T(n) = \begin{cases} O(n^d) & \text{if } k/b^d < 1 \\ O(n^d \log n) & \text{if } k/b^d = 1 \\ O(n^{\log_b k}) & \text{if } k/b^d > 1 \end{cases}$$

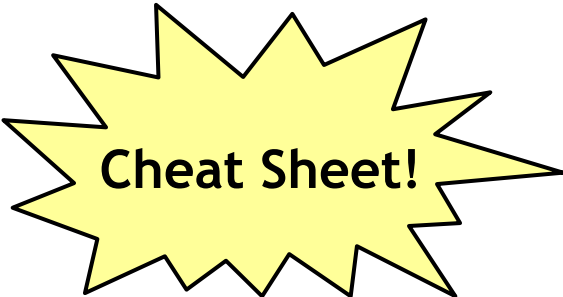
- ▶ **Remark:** If we replace O with Θ in the recurrence, then the closed form solution is tight
- ▶ Master Theorem also holds if the first term is of the form $kT\left(\left\lfloor \frac{n}{b} \right\rfloor\right)$ or $kT\left(\left\lceil \frac{n}{b} \right\rceil\right)$
- ▶ **Example:** For MERGESORT we have $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$
 - ▶ $\frac{k}{b^d} = \frac{2}{2^1} = 1$
 - ▶ By the Master Theorem: $T(n) = O(n \log n)$

Master Theorem with Log Factors

- The Master Theorem generalizes to recurrences with a log factor in the combination term

$$T(n) = kT\left(\frac{n}{b}\right) + O(n^d \log^w n) \quad \text{where } k \geq 1, b > 1, d \geq 0, w \geq 0.$$

$$T(n) = \begin{cases} O(n^d \log^w n) & \text{if } k/b^d < 1 \\ O(n^d \log^{w+1} n) & \text{if } k/b^d = 1 \\ O(n^{\log_b k}) & \text{if } k/b^d > 1 \end{cases}$$



Cheat Sheet!

Majority Elements (WS 2.1)

$$T(n) = \begin{cases} O(n^d) & \text{if } k/b^d < 1 \\ O(n^d \log n) & \text{if } k/b^d = 1 \\ O(n^{\log_b k}) & \text{if } k/b^d > 1 \end{cases}$$

Given an array A of n integers, a **majority element** of A is an element in A that appears **strictly more than $n/2$ times**, if such an element exists.

The algorithm MAJORITYELEMENT defined here finds the majority element of A if it exists, or return $\perp$ otherwise.

Give the runtime recurrence of MAJORITYELEMENT then find the asymptotic runtime using Master Theorem. (Assume COUNT takes linear time)

MAJORITYELEMENT($A[1, \dots, n]$):

if $n = 1$ **then return** $A[1]$

$m \leftarrow \lfloor n/2 \rfloor$

$x \leftarrow \text{MAJORITYELEMENT}(A[1, \dots, m])$

$y \leftarrow \text{MAJORITYELEMENT}(A[m + 1, \dots, n])$

if $x \neq \perp$ **then**

$c \leftarrow \text{COUNT}(x, A)$ //count occurrence of x in A

if $c > n/2$ **then return** x

if $y \neq \perp$ **then**

$c \leftarrow \text{COUNT}(y, A)$ //count occurrence of y in A

if $c > n/2$ **then return** y

return $\perp$

Majority Elements (WS 2.1)

Given an array A of n integers, a *majority element* of A is an element in A that appears **strictly more than $n/2$ times**, if such an element exists.

The algorithm MAJORITYELEMENT defined here finds the majority element of A if it exists, or return $\perp$ otherwise.

Prove that the algorithm is correct.

Hint: If z is a majority element of array A , is it necessary that z is also a majority element of both subarrays $A[1, \dots, \frac{n}{2}]$ and $A[\frac{n}{2} + 1, \dots, n]$? At least one?

MAJORITYELEMENT($A[1, \dots, n]$):

if $n = 1$ **then return** $A[1]$

$m \leftarrow \lfloor n/2 \rfloor$

$x \leftarrow \text{MAJORITYELEMENT}(A[1, \dots, m])$

$y \leftarrow \text{MAJORITYELEMENT}(A[m + 1, \dots, n])$

if $x \neq \perp$ **then**

$c \leftarrow \text{COUNT}(x, A)$ //count occurrence of x in A

if $c > n/2$ **then return** x

if $y \neq \perp$ **then**

$c \leftarrow \text{COUNT}(y, A)$ //count occurrence of y in A

if $c > n/2$ **then return** y

return $\perp$

Intermission

10 Tips to Succeed in EECS 376 (Part 1)

1

Definitions

definition 

/ˌdɛf əˈnɪʃ ən/ ☐ Phonetic (Standard) ☒ IPA

Make sure you understand (or at least have on your cheatsheet) the definitions of important concepts- they can be **good starting points** when you get stuck.

2

Take HWs Seriously



The HWs are designed to help **prepare you for the exam**. While they may be challenging, approach them sincerely and with dedication!

To be continued...

Divide and Conquer



Giving an Algorithm (Handout 3)

- ▶ On HWs, we will often ask you to “give an algorithm” for a certain problem. Here is what you’ll need to provide, unless otherwise mentioned
- ▶ **High-level English description** of the algorithm's **structure**, **main steps**, and what they do
- ▶ **Precise pseudocode** for the algorithm and any subroutines (helper functions)
 - ▶ The emphasis should be on the **clarity** and **readability** by humans (not "compilability" by a computer)
 - ▶ The pseudocode should be **high-level**, but **detailed** enough for a (good) programmer to implement in a real programming language based on your pseudocode
- ▶ **Analysis of the algorithm**
 - ▶ **Correctness analysis:** Prove that your algorithm outputs a **correct answer** on any (valid) input- often start by showing some "key property" of the problem that your algorithm exploit
 - ▶ **Runtime analysis:** Give an **asymptotic bound** on the (worst-case) running time as a function of input size (Usually a Big-O bound will suffice unless otherwise mentioned)

CBT Local Max (WS 2.2)

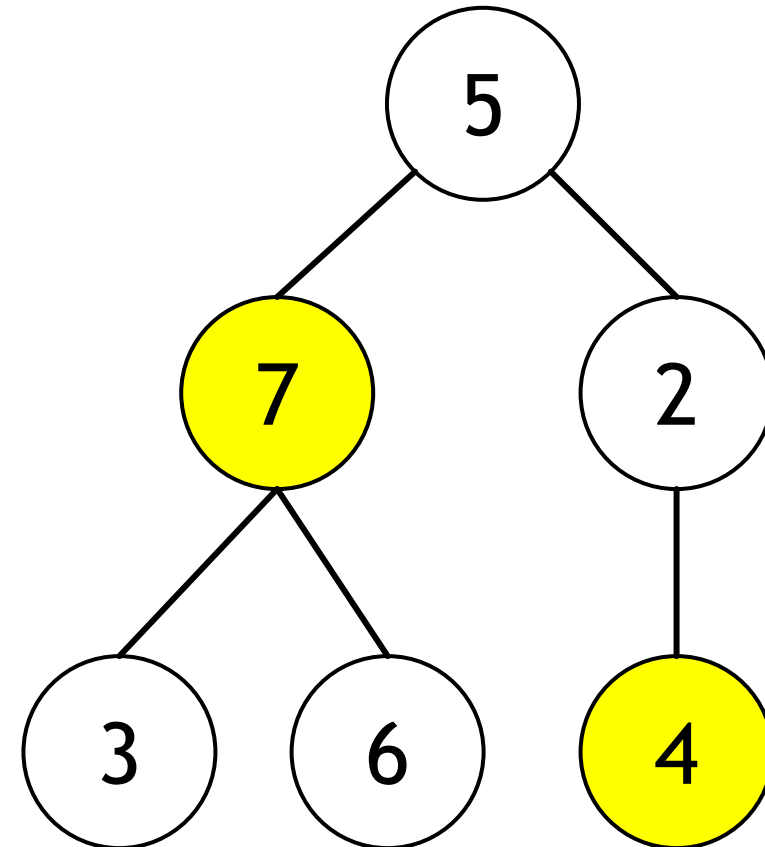
Consider a complete binary tree $T = (V, E, r)$ rooted at r where each vertex is labelled with a distinct integer. A vertex $v \in V$ is a *local maximum* if the label of v is **greater** than the label of each of its **parent and children**.

Give an algorithm that returns a local maximum of T using $O(\log|V|)$ vertex label queries.

Hint: For the recursive case, consider cases where:

- Root node > both children
- Root node < at least one of its children

Definition: A *complete binary tree* is a binary tree in which every level, except possibly the last, is completely filled and all nodes in the last level are **as far left as possible**.



Extra Slides

MERGESORT: Iterative Merge

MERGESORT ($A[1, \dots, n]$):

if $n = 1$ **then return** A

$m \leftarrow \lfloor n/2 \rfloor$

$L \leftarrow \text{MERGESORT}(A[1, \dots, m])$

$R \leftarrow \text{MERGESORT}(A[m + 1, \dots, n])$

return MERGE(L, R) // $O(n)$

MERGE ($L[1, \dots, \ell], R[1, \dots, r]$): // Iterative version

Initialize $p_L = p_R = 1$, empty array A

while $p_L \leq \ell$ **and** $p_R \leq r$ **do**

if $L[p_L] < R[p_R]$ **then**

append $L[p_L]$ to A

$p_L \leftarrow p_L + 1$

else

append $R[p_R]$ to A

$p_R \leftarrow p_R + 1$

if $p_L \leq \ell$ **then** append the remaining of L to A

else if $p_R \leq r$ **then** append the remaining of R to A

return A

MERGESORT: Tree Analysis (optional)

- ▶ Total Runtime = # recursive calls $\times$ work per recursive call
- ▶ Number of recursive call, d (or the “depth” of the tree)
 - ▶ Reach base case when the size of subproblem is 1
 - ▶ Size of subproblem is halved every recursive call
 - ▶ Solve for $\frac{n}{2^d} = 1 \Rightarrow d = \log_2 n = O(\log n)$
- ▶ Work per recursive call: $O(n)$
- ▶ Total runtime: $O(n) \cdot O(\log n) = O(n \log n)$

```
MERGESORT (A[1, ..., n]):  
    if n = 1 then return A  
    m  $\leftarrow$   $\lfloor n/2 \rfloor$   
    L  $\leftarrow$  MERGESORT(A[1, ..., m])  
    R  $\leftarrow$  MERGESORT(A[m + 1, ..., n])  
    return MERGE(L, R) // O(n)
```

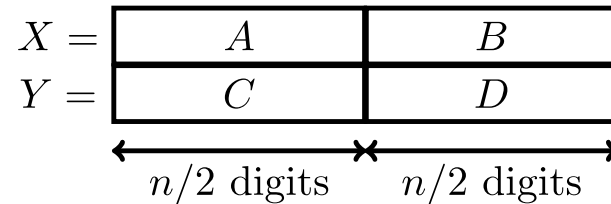
Karatsuba Algorithm: Big Idea

- ▶ We want to multiply two numbers X and Y . Each has n digits. Naïve way: $O(n^2)$

- ▶ Rewrite X and Y as follows:

- ▶ $X = A \cdot 10^{\frac{n}{2}} + B$

- ▶ $Y = C \cdot 10^{\frac{n}{2}} + D$



- ▶ Expand $X \cdot Y$ as follows:

- ▶ $X \cdot Y = \left(A \cdot 10^{\frac{n}{2}} + B \right) \left(C \cdot 10^{\frac{n}{2}} + D \right) = AC \cdot 10^n + (AD + BC) \cdot 10^{\frac{n}{2}} + BD$

- ▶ **Observation:** The multiplications AC, AD, BC, BD are **smaller versions of the original problem**

- ▶ We can use Divide and Conquer!

- ▶ **Karatsuba Algorithm:** Clever “preparations” to make the conquer step faster

Karatsuba Algorithm

- ▶ "Clever Preparations" in Karatsuba Algorithm
 - ▶ $M_1 = AC$
 - ▶ $M_2 = BD$
 - ▶ $M_3 = (A + B)(C + D)$
- ▶ Remember we wanted $AC \cdot 10^n + (AD + BC) \cdot 10^{\frac{n}{2}} + BD$
 - ▶ Algebra says $M_3 - M_1 - M_2 = AD + BC$
- ▶ **Runtime Recurrence:** $3T\left(\frac{n}{2}\right) + O(n) = O(n^{\log_2 3}) = O(n^{1.585})$
 - ▶ Better than $O(n^2)$

```
KARATSUBA(x, y): // Assume  $x \geq y$ 
     $n \leftarrow$  number of digits of  $x$ 
    if  $n = 1$  then return  $x \cdot y$ 
    write  $x$  as  $a \cdot 10^{n/2} + b$ 
    write  $y$  as  $c \cdot 10^{n/2} + d$ 
     $M_1 \leftarrow$  KARATSUBA( $a, c$ )
     $M_2 \leftarrow$  KARATSUBA( $b, d$ )
     $M_3 \leftarrow$  KARATSUBA( $a + b, c + d$ )
    return  $M_1 \cdot 10^n + (M_3 - M_1 - M_2) \cdot 10^{n/2} + M_2$ 
```

Karatsuba Algorithm: Exercise

Compute 37×76

- ▶ $n = 4$ (number of digits)
- ▶ $A = 3, B = 7, C = 7, D = 6$
- ▶ $M_1 = AC = 3 \cdot 7 = 21$
- ▶ $M_2 = BD = 7 \cdot 6 = 42$
- ▶ $M_3 = (A + B)(C + D) = (3 + 7)(7 + 6) = 130$
- ▶ $37 \times 76 = 21 \cdot 10^2 + (130 - 21 - 42) \times 10 + 42 = 2100 + 67 + 42 = 2812$

```
KARATSUBA( $x, y$ ): //Assume  $x \geq y$   
     $n \leftarrow$  number of digits of  $x$   
    if  $n = 1$  then return  $x \cdot y$   
    write  $x$  as  $a \cdot 10^{n/2} + b$   
    write  $y$  as  $c \cdot 10^{n/2} + d$   
     $M_1 \leftarrow$  Karatsuba( $a, c$ )  
     $M_2 \leftarrow$  Karatsuba( $b, d$ )  
     $M_3 \leftarrow$  Karatsuba( $a + b, c + d$ )  
    return  $M_1 \cdot 10^n + (M_3 - M_1 - M_2) \cdot 10^{n/2} + M_2$ 
```